

Smart Cache System

Predictive Pre-Warming, Traffic Anomaly Defense, Multi-Tier Hot/Warm Engine & Disk Persistence

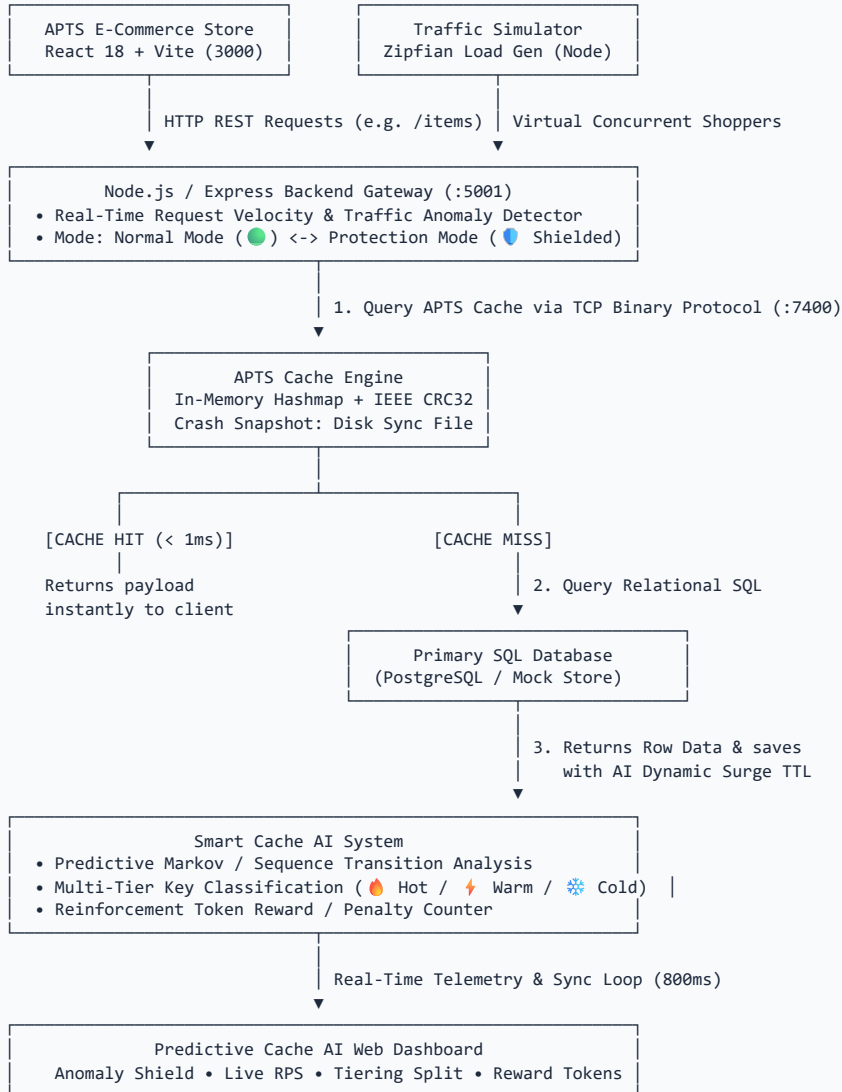
REPOSITORY	RUNTIME STACK	EVICTIOIN ENGINE	PERSISTENCE MODE
Smart-Cache-System	Node.js / React / TypeScript / SQL	Multi-Factor AI Utility Scoring	Crash-Resilient Disk Snapshot (JSON)

1. System Overview & Core Capabilities

The **Smart Cache System** is an enterprise-grade distributed caching engine engineered to solve high-concurrency latency bottlenecks, flash-sale spikes, and origin database exhaustion. Unlike standard reactive caching systems (e.g. Redis LRU/LFU) that only cache items after a slow database query has already occurred, this system implements an active **AI Decision & Prediction Layer** that predicts what will be requested next, protects the origin during traffic surges, and maintains crash-resilient disk snapshots.

- Key Innovations Implemented:**
- Traffic Anomaly Detection & Protection Mode** (Auto-scales TTLs 3x and shields origin database).
 - Crash-Resilient Disk Persistence** (Dual-memory architecture: in-RAM speed + periodic snapshot persistence to disk).
 - Multi-Tier Hot/Warm/Cold Key Classification** (Real-time velocity & acceleration tagging).
 - Multi-Factor AI Eviction Scoring** (Replaces naive LRU with continuous economic utility scoring).
 - Reinforcement Learning Token Ledger** (Reward tokens for prefetch hits with confidence-based policy fallback).

2. End-to-End Architectural Pipeline



3. Core Algorithms & Mathematical Formulations

1. Traffic Anomaly Detection & Origin Protection Mode

Anomaly Defense

The system maintains a rolling window of recent request arrival timestamps $\{t_1, t_2, \dots, t_k\}$. The current Requests Per Second (RPS) is computed continuously over a sliding 5-second window:

$$\text{RPS}_{\text{current}} = \frac{|\{t_i \mid \text{now} - t_i \leq 5\text{s}\}|}{5.0}, \quad \text{Anomaly Condition: } \text{RPS}_{\text{current}} > \max(20, 2.8 \times \text{RPS}_{\text{baseline}})$$

Automatic Defense: When an anomaly (e.g. 10x Spike Test) triggers, the system flips to **PROTECTION MODE**. Cache TTLs are immediately multiplied by $\times 3$ (180s to 540s), and aggressive pre-warming is initiated to shield the origin database. When traffic normalizes below 12 RPS for $\geq 12\text{s}$, the system reverts to **NORMAL MODE**.

2. Multi-Factor AI Eviction Scoring Engine

Cache Optimization

Traditional caches rely on naive single-variable LRU (evict oldest) or LFU (evict lowest frequency). The Smart Cache Engine replaces this with a continuous multi-factor AI Utility Score:

$$\text{Score}(k) = 0.4 \times \text{Hotness}(k) + 0.3 \times \text{Recency}(k) + 0.2 \times \text{Frequency}(k) - \text{SizePenalty}(k)$$

Where $\text{Recency}(k) = 1 / (1 + \ln(1 + \Delta t_{\text{lastAccess}}))$, $\text{Frequency}(k) = \min(1.0, \text{hits}/25)$, and $\text{SizePenalty}(k) = \min(0.2, \text{size}/512\text{KB})$. Under memory pressure ($> 90\%$), the engine evicts the key with the lowest AI score rather than a frequently used item that was momentarily idle.

3. Crash-Resilient Disk Snapshot Persistence

Data Resilience

Standard in-memory caches lose all cached data upon process termination or RAM crash. The APTS Cache Engine implements dual-tier persistence:

- In-RAM Fast Path:** Serves lookups and updates directly from high-speed memory maps ($< 1\text{ms}$).
- Atomic Disk Snapshots:** A background daemon serializes non-expired keys, values, and remaining TTLs to `Cache_Engine/data/cache-snapshot.json` every 10 seconds and upon process signals (SIGINT, SIGTERM).
- Zero-Downtime Recovery:** On boot, the engine detects and parses the snapshot, restoring hot and warm keys into RAM before receiving traffic.

4. Hot / Warm / Cold Tiering Classification

Demand Forecasting

Every key in the cache engine is continuously scored based on its request velocity V (hits / age in seconds) and acceleration A ($\Delta V / \Delta t$):

$$\text{Hotness}(k) = \min\left(1.0, \left(V \times 2.0 + \frac{\text{hits}}{10}\right) \times \frac{1}{1 + \log_{10}(\Delta t_{\text{recency}} + 1)}\right)$$

Keys are partitioned into: 🔥 **HOT** ($\text{Hotness} \geq 0.5$) or ($\text{hits} \geq 5$), ⚡ **WARM** ($\text{Hotness} \geq 0.2$) or ($\text{hits} \geq 2$), and ❄️ **COLD** ($\text{Hotness} < 0.2$).

5. Reinforcement Token Reward & Confidence Controller

Self-Learning Policy

The system tracks prediction quality via a reinforcement token ledger:

- +10 Tokens:** Awarded when a pre-cached key generates a confirmed cache hit (true positive).
- 5 Tokens:** Deducted if a pre-cached key expires without ever receiving a hit (false positive).
- Confidence Fallback:** If AI confidence ($\text{Acc} < 60\%$), the system falls back to conservative LRU/LFU caching. Once confidence exceeds (60%) , adaptive predictive caching takes control.

4. Codebase Architecture & Key Files

Component	File Location	Implemented Advanced Algorithms
Cache Server	Cache_Engine/cache-server.js	Disk snapshot persistence, Hot/Warm/Cold tiering, AI eviction scoring, CRC32, TCP :7400, HTTP :7401.
Disk Snapshot Store	Cache_Engine/data/cache-snapshot.json	Serialized disk snapshot restoring RAM cache on startup.
AI Engine & Orchestrator	website1/backend/src/cache/predictive-engine.ts	Traffic Anomaly Detector, Protection Mode, Hotness & Demand forecasting, RL Token Ledger, Markov lookahead.
Cache TCP Driver	website1/backend/src/cache/apts-client.ts	Binary wire protocol client framing commands over raw sockets.
Backend API Core	website1/backend/src/server.ts	Fastify/Express routing, telemetry API, dashboard handler.
Live Dashboard	PredictiveCache-AI-Package/PredictiveCache-AI-Dashboard.html	Real-time operations center with Anomaly Shield pill, Tiering counters, Disk snapshot badge, and Reward tokens.
Storefront App	website1/frontend/src/App.tsx	React single-page application connecting customer interactions to cache layer.
Traffic Simulator	website1/traffic-simulator/src/simulator.ts	Zipfian load generator triggering burst tests and steady loads.

5. How to Demonstrate in a Viva / Presentation

- Start all services:** Run `.\start-all.ps1` from the root directory.
- Verify Disk Snapshot Recovery:** Check the Cache Engine terminal. It will print: 📁 [Persistence Engine] Restored X keys from disk snapshot. Show `Cache_Engine/data/cache-snapshot.json`.
- Demonstrate Traffic Anomaly & Protection Mode:** Open the dashboard at `http://localhost:5001/dashboard`. Click "⚡ 10x Spike Test". Point out how the Anomaly Detection pill turns into: 🛡️ PROTECTION MODE ACTIVE (Spike Shielded) and scales dynamic TTLs to protect the database.
- Show Multi-Tier Hot/Warm/Cold Keys:** Point to the tiering counter in the dashboard header: 🔥 Hot Keys | ⚡ Warm Keys | ❄️ Cold Keys updating in real time.
- Show Reinforcement Tokens:** Show the 🟡 Reward Tokens pill accumulating tokens as user clicks hit pre-warmed cache.